

■ Agentic Design Patterns 开源书小白指南

Antonio Gulli 《Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems》——424 页开源 + 全套 Jupyter notebooks, Agent 时代的"GoF 设计模式"

2025-09 开源 · 21 个设计模式 + 7 个附录 · 三大框架画布 (LangChain/LangGraph · CrewAI · Google ADK) · 拾遗任务 f5b25c7d

0 · 这本书是什么：一句话与来龙去脉

一句话：它是 Agent 领域的《设计模式》教科书——就像 1994 年 GoF 把面向对象编程里反复出现的解法命名成"单例、观察者、工厂"一样，Google 高级工程师 Antonio Gulli 把 **构建 AI Agent 时反复出现的 21 种结构解法** 逐一命名、画图、配代码。

三个硬核事实：

- **整本开源免费**：PDF (实际 482 页) + 配套 notebooks 全部公开，GitHub 镜像仓库 850+ star，另有中文翻译版《智能体设计模式》；
- **每个模式都有可运行代码**：50+ 个 notebook，横跨 LangChain/LangGraph、CrewAI、Google ADK 三套框架——同一模式三画布对照；
- **Google 官方背书**：Google Cloud 官方架构文档《Choose a design pattern for your agentic AI system》直接引用这本书的模式分类。

✦ **版权与依赖提示**：书免费开源，但 notebook 全部依赖 `OPENAI_API_KEY` 或 `GOOGLE_API_KEY` (Gemini / GPT / Vertex AI)，跑原版 notebook 需自备 Key。本指南实验用"确定性 FakeLLM"离线复现模式骨架，模式代码一行不改即可换成真模型。

疑问 1 · 424 页讲 21 个模式，全景长什么样？怎么分层记住？

把 21 章按"解决什么问题"分成四层，一张图记住全书：

第一层·基础编排（单 Agent 怎么干活）

1 提示链

2 路由

3 并行化

4 反思

5 工具使用

6 规划

第二层·协作与通信（多个 Agent 怎么配合）

7 多智能体协作

8 记忆管理

9 自适应改进

10 MCP

15 A2A 通信

第三层·可靠性与安全（怎么不出事）

11 目标设定监控

12 异常处理恢复

13 人在回路

14 RAG

18 护栏

第四层·元能力与工程化（让 Agent 变强、变省、被度量）

16 资源感知优化

17 推理技术

19 评估监控

20 优先级

21 探索发现

记忆口诀：单兵 → 团队 → 安全带 → 教练。第一层是单 Agent 的六种基本功，第二层是组队的五种协作方式，第三层是别翻车的五条安全带，第四层是让系统持续变强的五种元能力。

疑问 2 · Level 0~3 复杂度分级是什么？我的项目在第几级？

自主性与能力逐级叠加：每一级 = 上一级 + 新模式

Level 0 裸脑
纯 LLM 问答

Level 1 连接者

+ 工具/RAG/搜索
会查资料再回答

Level 2 战略家

+ 规划/反思/记忆
多步自主循环
上下文工程

Level 3 协作系统

+ 多 Agent 协作
A2A 通信 / 编排
自改进 · 探索发现

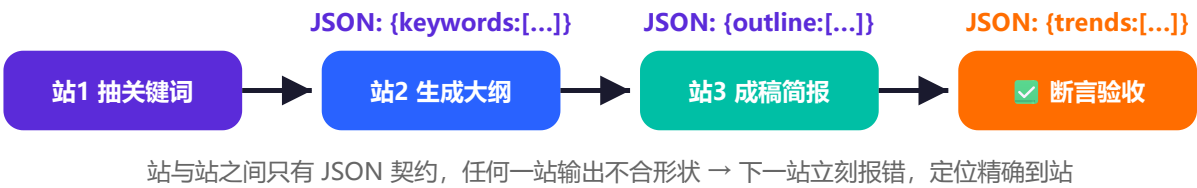
Level 0 = 裸 LLM（只会背训练数据，问它昨天新闻必瞎编）→ **Level 1 = 接上工具**（对应模式 5/14）→ **Level 2 = 会规划会反思**（模式 4/6/8/17）→ **Level 3 = 多 Agent 协作系统**（模式 7/15/21）。这本书的模式地图就是一条"升级路线图"：想从 Level 1 升 Level 2，就补反思+规划+记忆三个模式。

疑问 3 · 提示链和我已会的 Agent 循环，本质区别是什么？

知识库里有 [[Agent循环]] (模型↔工具自动转圈到任务完成)。提示链是它的**反面兄弟：不给自主权**。

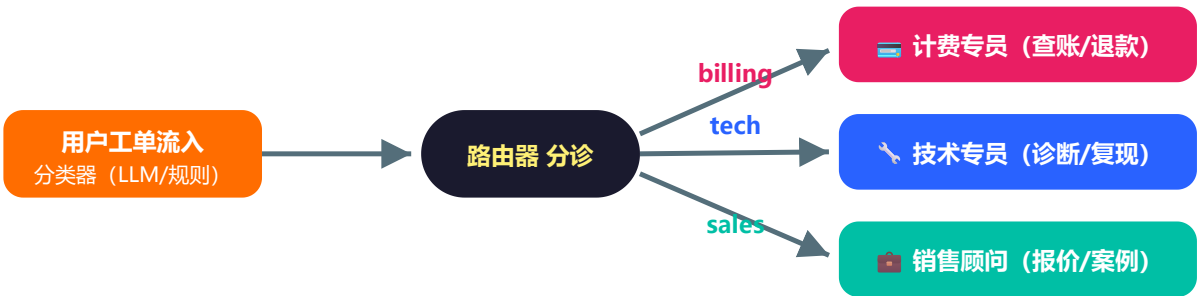
	提示链 Prompt Chaining (Ch1)	Agent 循环 (已有概念)
控制流	人写死的固定流水线，站与站顺序执行	模型自己决定下一步，循环到完成
交接物	站间 JSON 契约，类型固定可断言	自然语言 + 工具调用结果
确定性	高：每站可单测，坏在哪一站立刻知道	低：路径不可预测，调试靠 trace
适用	流程稳定、步骤明确的任务（翻译→摘要→审校）	开放式任务（"帮我调研 X"）

选型法则：能用提示链解决就别上 Agent 循环——确定性高、省 token、可测试；只有步骤本身需要模型临场决定时才升级为循环。



疑问 4 · 路由模式：分诊台是怎么搭的？

路由 (Ch2) = 前面放一个轻量分类器，后面挂 N 个专科工位。每条输入只走一条路——省 token、且每条路可独立测试。书中三画布实现：LangGraph 用 **条件边** (知识库 [[LangGraph与Agent编排]] 已学)、CrewAI 用 hierarchical 流程、ADK 用 agent transfer。



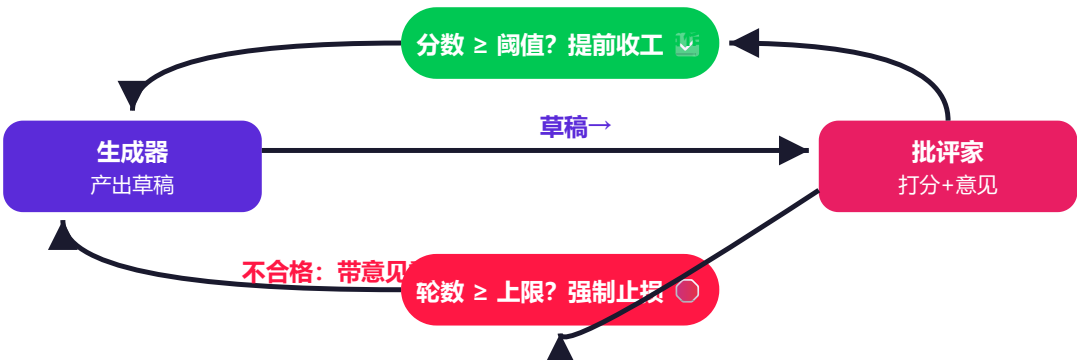
与已有概念的连接：[[技能路由器与授权硬门]] 是路由模式的"技能选择"特例；[[能力层与后端路由]] 是"后端选择"特例。书的贡献是给出 **通用命名 + 三框架对照实现**——模式一旦命名，就能跨项目复用。

疑问 5 · 并行化的三种形态：切片、投票、总裁

形态	做法	生活类比	典型场景
Sectioning 切片	任务切成独立子块并行，结果拼接	教材分章多人同写	长文分节摘要、多文件审查
Voting 投票	同一任务跑 N 遍，取多数/最优	三位评委独立打分	降低幻觉、提高判断可靠性
Governing 总裁	多视角并行给意见，一个总裁合成	各部门汇报，总裁拍板	多维度审稿：事实+文风+安全

Voting 的本质是花 3 倍 token 买确定性——与 [[LLM 采样与温度]] 的知识呼应：温度 > 0 的模型单次输出有随机性，多次投票用冗余换稳定。Governing 则是 [[多智能体协作]] 的最轻量前身：不用完整 Crew，一个 ThreadPoolExecutor 就够（见实验模式 3）。

疑问 6 · 反思循环：为什么必须有两个停止条件？



只有"分数达标才停"一个条件 → 碰上永远不合格的任务就无限烧钱；只有"最多 N 轮"一个条件 → N 轮后拿到的可能是没达标的半成品且无人知晓。两个一起才是安全设计：阈值管**质量**，轮数上限管**成本**。知识库对照：反思模式就是 [[质检Gate与自我纠错循环]] 的 LLM 生成侧版本——Gate 的"打分器必须确定性"在真模型场景松动了（LLM 当批评家本身也有随机性），所以书里 Ch19 干脆再加一层 **LLM-as-a-Judge 的评估**来校准批评家自己。

疑问 7 · 我知识库完全空白的新大陆：六个冷门模式速览

模式	一句话	与已有知识的关系
11 目标设定与监控	给 Agent 定可度量的目标并持续追踪偏差，而不是让它"看着办"	[[工单任务图与前沿调度]] 的"验收标准"前移为目标本身
12 异常处理与恢复	工具失败/超时/格式错的 fallback 链与降级预案	[[数据源降级链]] 的通用版
15 A2A 智能体通信	Agent 间互发现、互调用的协议 (AgentCard 名片)	[[MCP协议]] 连"Agent↔工具", A2A 连"Agent↔Agent", 互补
16 资源感知优化	Agent 知道自己花多少钱/多少 token, 按预算选模型选工具	[[上下文预算与战略压缩]] 的花钱自觉版
20 优先级 SuperSimplePM	用产品经理的 RICE 思维给任务排序, Agent 先做"该做的"而非"能做的"	[[指令优先级层级]] 的任务侧兄弟
21 探索与发现	Agent Laboratory: 文献调研→假设→实验→论文的全自动科研循环	[[AgentHarness智能体挽具]] 的极限应用场景

疑问 8 • 护栏模式和提示注入防线是什么关系？

护栏 (Ch18) 是**输入侧 + 输出侧的双重安检门**：

- **输入护栏**：敏感词过滤、注入检测、权限校验——对接已有 [[提示注入]] 三防；
- **输出护栏**：Pydantic schema 校验、工具名白名单、置信度封顶——对接 [[Agent输出协议契约]] 的"协议门拒收坏结果"；
- **LLM 当护栏**：再放一个模型当安检员读输出找违规——快但自身有漏报，所以 **确定性校验永远在 LLM 护栏外面包一层**（代码校验是底线，模型校验是补充）。

已有 [[评分契约与护栏]] 是护栏模式在金融评分域的实例；书把它升格为通用安全模式，和评估 (Ch19)、人在回路 (Ch13) 组成"安全三件套"。

疑问 9 • LLM-as-a-Judge 自动评估怎么工作？可信吗？

Ch19 的核心：写一份 **评分 rubric (评分细则)** 喂给一个强模型，让它按细则给被测 Agent 的输出打分——相当于雇了一个不知疲倦的外部裁判。书中示例是法律综述质量 rubric：论据是否可溯源、结构是否完整、术语是否准确，逐项打分。

可信度三定律（书上+社区共识）：

- **细则质量 > 裁判模型强度**：rubric 模糊，GPT-4o 级裁判也打糊涂分；
- **裁判不能只评自家**：同家族模型自评有偏，最好跨家族交叉；
- **人抽检不可省**：定期抽样人工比对，防裁判漂移——就是 [[证据优先质检ProofOverClaims]] 的"看实机画面"精神。

疑问 10 · 配套 notebooks 怎么用？没有 API Key 怎么学？

仓库结构：PDF 一本 + `notebooks/` 下 61 个文件（覆盖 21 章 + 附录 A-G，多为 Colab 导出的无扩展名 ipynb）。三条学习路线：

路线	做法	成本
① 真跑原版	申请 Gemini API Key（有免费额度）→ Colab 直接跑	有 Key 才行
② 读代码学结构	notebook 就是“模式 + 三框架对照”的代码卡，读源码足以理解模式骨架	零成本
③ 离线复现（本指南采用）	用 FakeLLM 把模式骨架跑起来，验证控制流与停止条件	零成本、确定性

附录 A-G 也别错过：A 高级提示技巧（28 页最厚）、F 推理引擎内幕、G Coding agents——本身是六篇独立小专题。

⚡ 动手实验：FakeLLM 离线复现 5 大模式（已真跑通过）

实验目标：用纯 Python + 确定性 FakeLLM 一次跑通书中 5 个代表性模式（提示链 / 路由 / 并行化 / 反思 / 护栏），验证每个模式的控制流和停止条件。环境：Python 3.14 + pydantic，**零 API 依赖，可无限次重复，结果完全一致。**

1 进入实验目录并运行

```
cd F:\reminder\agentic-design-patterns\exercise
set PYTHONUTF8=1 && python patterns_playground.py
```

目的：跑通五模式流水线。✅ 预期：连续输出五个模式区块，最后一行 `ALL 5 PATTERNS PASSED`。

2 看模式 1 提示链输出：输入 "Agent Memory Routing"，三站依次产出 `{"keywords":[...]}` → `{"outline":[...]}` → `{"trends":[...]}`，末行断言 `trend_name=="agent"` 通过。✅ 预期：三站 JSON 形状逐站变化但全程合法——固定流水线的可断言性。

3 看模式 3 并行化输出：三个 Critic（事实核查/文风审查/安全审查）在 `ThreadPoolExecutor` 里真并发，总裁一次合成。✅ 预期：三条 critic 意见全部出现（"投诉率为0不可信"、"绝对化表述"、"放行"）。

4 看模式 4 反思循环输出：文案三轮打磨，4 分→7 分→9 分，第 3 轮达标 **early-stop**（省下 2 轮调用）。✅ 预期：逐轮分数递增并提前停机。

5 看模式 5 护栏输出（最有含金量）：越权工具名 `delete_db` 被白名单正则拒收；置信度 1.7 被 `Field(ge=0,le=1)` 拒收；第三次合法调用放行。✅ 预期：两次 ❌ 拒收 + 一次 ✅ 放行——**坏输出根本进不了执行层**，且拒收理由（`ValidationError` 文本）可直接回流给模型重试。

6 改造玩：把 `THRESHOLD=8` 改成 `11` 再跑——反思循环打不到阈值，改由最大轮数保险丝兜底止损。亲身体会“两个停止条件缺一不可”。

✓ 实测结果：5 个模式全部 PASS，重复运行输出完全一致（确定性验证）；真实运行完整输出存于 `exercise\run_output.txt`。

诚实记录：原版 61 个 notebook 依赖 `OPENAI_API_KEY` / `GOOGLE_API_KEY`（本机未配置 Key），未真跑原版；本实验复现的是模式控制流骨架，将 FakeLLM 换成真模型客户端即为书中原版做法。

这本书给我什么：能力清单

- **一张模式地图**：21 个模式四层分类 + Level 0-3 升级路线，拿到任何 Agent 项目先对号入座；
- **一套选型判断力**：提示链 vs 循环、投票 vs 单次、LLM 护栏 vs 代码护栏——每对都给出取舍条件；
- **三框架对照视野**：同一模式在 LangChain/LangGraph、CrewAI、ADK 的三种写法，框架迁移不再恐慌；
- **安全与评估补课**：护栏、异常恢复、LLM-as-a-Judge、目标监控——应用层最容易漏的四件事；
- **离线学习方法**：FakeLLM 确定性复现模式骨架，这套方法可复用到任何"代码依赖 API"的开源书学习。